

JavaScript vs ECMA(European Computer Manufacturers Association) Script

ECMAScript 는 JavaScript 표준

<https://ecma-international.org/publications-and-standards/standards/ecma-262/>

Javascript는 2011년에 정의한 ES5, ES6 ~ ES16 (ES2025)

ES6 버전 으로 작성된 JS 코드는 브라우저 호환성 문제가 있음

: OLD 브라우저 에서는 ES6 버전 이상의 JS 코드는 해석을 할 수 없는 문제가 있음

ES6+ 호환성 테이블 표

<https://compat-table.github.io/compat-table/es6/>

Babel js 컴파일러 (변환)

바벨 <https://babeljs.io/> , SWC <https://swc.rs/>

: ES6+ 코드를 ES5 버전으로 변환해서 브라우저 호환성을 유지해야 함

Bundler - WebPack <https://webpack.js.org/>, RollUp <https://rollupjs.org/>

: Module Bundler (여러개의 js, css 파일들을 1개의 js, css 파일로 합쳐 주는 역할)

: npm 도구를 사용해서 JS 라이브러리 설치하면 node_modules 폴더가 생성

ESLint <https://eslint.org/>

: 코드 정적 분석 도구

Build 도구의 필요성에 대한 질문

<https://gemini.google.com/share/67207e22daf7>

JavaScript Build Tools

CRA(Create-React App) <https://create-react-app.dev/>

Vite <https://vite.dev/>

: 빠른 HMR(Hot Module Replacement) 기능 제공

: 모듈 번들러 Rollup을 사용함 (<https://rollupjs.org/>)

TypeScript <https://www.typescriptlang.org/>

: ECMAScript + Type

ReactJS <https://react.dev/> - CSR(Client Side Rendering) - SEO(검색엔진최적화) 되어 있지 않음

NextJS <https://nextjs.org/> - CSR / SSR(Server Side Rendering) 방식 모두 지원함

Ajax (Asynchronous Javascript and Xml)

jQuery <https://jquery.com/>

NodeJS, NPM(node package manager), package.json 파일의 역할

각각의 역할을 이해하기

Nodejs 21 버전 설치 후에

node -v

v21.7.3

npm -v

JavaScript Code

FormData 객체 <https://developer.mozilla.org/ko/docs/Web/API/FormData>

//FormData 간단 예제

```
const formData = new FormData();
formData.append('key1', 'value1');
formData.append('key2', 'value2');
```

// 방법 1: entries() 사용

```
for (let [key, value] of formData.entries()) {
  //console.log(key, value);
}
```

// 방법 2: forEach 사용

```
formData.forEach((value, key) => {
  console.log(key, value);
});
```

JavaScript의 FormData 객체 설명

FormData는 HTML 폼(form) 데이터를 쉽게 구성하고 전송할 수 있도록 도와주는 JavaScript

객체입니다. 주로 AJAX 요청(XHR, Fetch API 등)과 함께 사용되어 서버로 폼 데이터를 전송할 때 유용합니다.

기본 특징

1. 키-값(key-value) 쌍으로 데이터 저장
2. 멀티파트 폼 데이터(파일 업로드) 지원
3. 폼 요소와 연동 가능
4. 서버로 전송하기 쉬운 형식 제공

기본 사용법

1. 폼 요소로부터 생성

```
const formElement = document.querySelector('form');
const formData = new FormData(formElement);
```

2. 빈 객체 생성 후 데이터 추가

```
const formData = new FormData();
formData.append('username', 'john');
formData.append('avatar', fileInput.files[0]);
```

● 메서드

● 설명

append(name, value)

새로운 키-값 쌍 추가

<code>append(name, blob, filename)</code>	파일 데이터 추가
<code>delete(name)</code>	특정 키 삭제
<code>get(name)</code>	첫 번째 값 가져오기
<code>getAll(name)</code>	모든 값 가져오기 (동일한 이름의 여러 입력 필드)
<code>has(name)</code>	키 존재 여부 확인
<code>set(name, value)</code>	기존 값 수정 또는 새 값 추가
<code>entries()</code>	모든 키-값 쌍을 순회 가능한 형태로 반환

사용자정의 객체선언 (Object Literal)

```
const user = {
  name: '홍길동',
  age: 30,
  greet: function() {
    console.log('Hello js');
  }
};
console.log(user);
//user object의 property를 읽어오기
console.log(user.name);
console.log(user.age);

//user object에 새로운 property 추가하기
user['addr'] = '집주소';
console.log(user);

//user object에 포함된 함수 호출하기
user.greet();

//사용자정의 객체선언
const user = {
  //property
  name: '홍길동',
  age: 30,
  //function
  greet: function() {
    console.log('Hello js');
  },
};
```

```

phone: {
  home: 123,
  office: 456
}
};
console.log(user);
//user object의 property를 읽어오기
console.log(user.name);
console.log(user.age);

//user object에 새로운 property 추가하기
user['addr'] = '집주소';
console.log(user);
//user object에 포함된 함수 호출하기
user.greet();

// ?. 연산자
//console.log(user.phone1.home);
/*
  "TypeError: Cannot read properties of undefined (reading 'home')
*/
console.log(user.phone1?.home);
// undefined

```

?? Nullish Coalescing

<https://www.daleseo.com/js-nullish-coalescing/>

falsy 인 경우 (6가지)

undefined, null, false, 0, "", NaN

//?? 연산자는 좌항이 undefined와 null인 우항을 선택하며,
//나머지는 항상 좌항을 선택합니다.

```

const user = {
  name: "김코딩",
  age: 0, // 의도적으로 0을 할당
  address: null
};

console.log(user.name ?? "이름 없음"); // "김코딩"
console.log(user.age || 25); // 25
console.log(user.age ?? 25); // 0 (의도된 0 값 유지)

console.log(user.address ?? "주소 미입력"); // "주소 미입력"
console.log(user.phone ?? "연락처 없음"); // "연락처 없음" (undefined)

```


ajax 통신 - fetch() 함수

XMLHttpRequest 객체

<https://developer.mozilla.org/ko/docs/Web/API/XMLHttpRequest>

XMLHttpRequest (초기 방법)

- 복잡하고 어려운 문법
- 콜백 지옥 발생
- 에러 처리 복잡
- 코드가 길고 읽기 어려움

fetch() 함수

https://developer.mozilla.org/ko/docs/Web/API/Fetch_API/Using_Fetch

Promise 객체

https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Global_Objects/Promise

fetch() (현대적 방법)

- 간단하고 직관적
- Promise 기반
- async/await 사용 가능
- 깔끔한 코드

기본 패턴

fetch(주소) - 요청 보내기

.then() - 응답 받기

.json() - JSON 데이터로 변환하기

.then() - 데이터 사용하기

/ 설명: - fetch()는 Promise를 반환해요*

- .then()으로 성공했을 때 실행할 코드 작성

- .catch()로 실패했을 때 실행할 코드 작성

- response.json()으로 JSON 데이터로 변환

**/*

```
fetch('https://jsonplaceholder.typicode.com/posts/1') // 데이터 요청
```

```
.then(response => response.json()) // JSON으로 변환
```

```
.then(data => {
```

```
  // 데이터 사용
```

```
  console.log('받은 데이터:', data);
```

```
  document.getElementById('result').innerHTML = data.title;
```

```
})
```

```
.catch(error => {
```

```
  // 에러 처리
```

```
  console.error('에러 발생:', error);
```

```
});
```

```

/* async/await 방식 ES7 버전에서 추가됨
- .then() 체이닝 없이 깔끔한 코드
- 비동기 함수 이지만 일반적인 동기 함수처럼 읽기 쉬움
- 에러 처리도 try/catch로 간단
- 디버깅하기 쉬움
*/
async function getData() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');
    const data = await response.json();
    console.log('데이터 받았어요!', data);
    return data;
  } catch (error) {
    console.error('에러 발생:', error);
  }
}
// 사용하기
getData();

```

JSON.stringify () 함수

https://developer.mozilla.org/ko/docs/Web/JavaScript/Reference/Global_Objects/JSON

JavaScript 객체나 값을 JSON 문자열로 변환합니다.

- 객체 → JSON 문자열 변환
- 함수나 Symbol 값은 변환에서 제외됨
- 순환 참조(자기 자신을 참조하는 객체)가 있으면 에러 발생

```

const person = {
  name: "홍길동",
  age: 30,
  hobbies: ["독서", "여행"],
  isStudent: false
};

// 객체를 JSON 문자열로 변환
const jsonString = JSON.stringify(person);
console.log(jsonString);
// 출력: {"name":"홍길동","age":30,"hobbies":["독서","여행"],"isStudent":false}

// 예쁘게 출력 (들여쓰기 적용)
const prettyJson = JSON.stringify(person, null, 2);
console.log(prettyJson);
/*
출력:
{
  "name": "홍길동",
  "age": 30,
  "hobbies": [
    "독서",
    "여행"
  ],
  "isStudent": false
}
*/

```

*/

JSON.parse()

JSON 문자열을 JavaScript 객체로 변환합니다.

- JSON 문자열 → JavaScript 객체 변환
- 유효하지 않은 JSON 문자열을 파싱하면 에러 발생

```
const jsonStr =
'{"name": "홍길동", "age": 30, "hobbies": ["독서", "여행"], "isStudent": false}';

// JSON 문자열을 객체로 변환
const obj = JSON.parse(jsonStr);
console.log(obj.name); // 출력: "홍길동"
console.log(obj.hobbies); // 출력: ["독서", "여행"]
console.log(obj.isStudent); // 출력: false
```

<https://developer.mozilla.org/ko/docs/Web/API/Document/querySelector>

event.preventDefault() 함수 설명

JavaScript에서 이벤트 객체의 메서드로, 해당 이벤트의 기본 동작을 취소하는 역할을 합니다.

주된 역할:

1. 기본 브라우저 동작 방지
 - 폼 제출(submit) 시 기본적으로 브라우저는 페이지를 새로 고침하거나 지정된 URL로 데이터를 전송합니다.
 - preventDefault()를 사용하면 이 기본 동작이 막혀 페이지 새로 고침 없이 JavaScript만으로 폼 처리가 가능해집니다.
2. 비동기 처리 가능
 - 주로 AJAX(예: fetch, axios)와 함께 사용되어 서버와의 통신을 백그라운드에서 처리할 수 있습니다.

```
document.querySelector("form").addEventListener("submit", function(event) {
  event.preventDefault(); // 폼 제출 시 페이지 새로 고침 방지
  // 여기서 AJAX 요청 또는 유효성 검사 수행
});
```

다른 활용 사례:

- <a> 태그의 클릭 시 링크 이동 차단
- 키보드 이벤트(keydown)에서 특정 키의 기본 동작 차단 (예: Enter 키로 폼 제출 방지)

ECMAScript Code

ECMA International

<https://ecma-international.org/publications-and-standards/standards/ecma-262/>

Browser ES6+ Support 현황

<https://compat-table.github.io/compat-table/es6/>

Javascript 호이스팅(Hoisting)에 대한 질문

<https://gemini.google.com/share/ab7dd1a40edd>

Arrow 함수의 this에 대한 질문 과 find() 함수에 질문

<https://gemini.google.com/share/4f6227295880>

ECMAScript Module에 대한 질문

<https://gemini.google.com/share/ad7a8fad26f3>

Template Literals

// `` back tick Template Literals

```
const port = 8080;
```

```
const domain = 'mydomain';
```

```
const url = 'http://' + domain + ':' + port;
```

```
console.log(url);
```

```
const url_after = `http://${domain}:${port}`;
```

```
console.log(url_after);
```

Arrow function (화살표 함수)

```
function add(n1,n2) {
```

```
  return n1 + n2;
```

```
}
```

```
console.log(add(10,20));
```

```
const add2 = (n1,n2) => n1 + n2;
```

```
console.log(add2(10,20));
```

// () 묵시적으로 return 구문을 포함한다.

```
const add3 = (n1,n2) => (n1 + n2);
```

```
console.log(add3(10,20));
```

```
const add4 = (n1,n2) => {
```

```
  let temp = n1 + 10;
```

```
  return temp + n2;
```

```
}
```

```
console.log(add4(10,20));
```

Destructuring Assignment

```
const user = {
  name:'둘리',
  age:30,
  phone: {
    home: '1234',
    mobile: '4566'
  }
};
console.log(user);
console.log(user.name);
console.log(user.age);
console.log(user.phone.home);
console.log(user.phone.mobile);
```

```
//2번에 나누어서 할당하기
const {name,age,phone} = user;
console.log(name);
console.log(age);
console.log(phone);
const {home,mobile} = phone;
console.log(home);
console.log(mobile);
```

```
//1번에 할당하기
const {name,age,phone:{home,mobile}} = user;
console.log(name);
console.log(age);
console.log(home);
console.log(mobile);
```

```
//객체에 속한 함수를 assign 받기
const obj = {
  names: 'React',
  greet() {
    return this.names;
  }
};
```

```
console.log(obj.greet()); //"React" (잘 출력됨)
//destructuring assignment 방식으로 함수 할당받기
const {names, greet} = obj;
console.log(names); //"React"
console.log(greet()); //undefined
```

```
//해당객체의 레퍼런스를 bind() 함수로 전달해주기
const boundGreet = greet.bind(obj);
console.log(boundGreet()); //"React"
```

```
//this 대신 객체 변수를 사용
const obj = {
  names: 'React',
  greet() {
    return obj.names;
  },
};
```

```

greet2: () => {
  return obj.names;
}
};
const {names,greet,greet2} = obj;

console.log(names);
console.log(greet());
console.log(greet2())

```

fetch() 함수와 Promise 객체 간단 설명

Promise (프로미스)

- "약속 상자" 같은 개념입니다.
- 비동기 작업(시간이 걸리는 작업)의 미래 결과값을 담을 수 있는 객체입니다.
- 3가지 상태가 있습니다:
 1. 대기 (pending): 아직 결과가 안 나온 상태
 2. 이행 (fulfilled): 성공적으로 완료된 상태
 3. 거부 (rejected): 실패한 상태

```

const myPromise = new Promise((resolve, reject) => {
  // 비동기 작업 수행
  if(성공) resolve(결과값); // 성공 시
  else reject(에러);        // 실패 시
});

myPromise
  .then(result => console.log(result)) // 성공 시 실행
  .catch(error => console.error(error)); // 실패 시 실행

```

fetch() 함수

- 웹에서 데이터를 가져오는 현대적인 방법입니다.
- 서버에 HTTP 요청을 보내고, **Promise**를 반환합니다.
- 기본적으로 GET 요청을 보냅니다.

```

fetch('https://api.example.com/data')
  .then(response => response.json()) // 응답을 JSON으로 변환
  .then(data => console.log(data))   // 실제 데이터 사용
  .catch(error => console.error('Error:', error)); // 에러 처리

```

왜 같이 사용하나요?

- **fetch()**는 네트워크 요청이라는 비동기 작업을 수행하므로 **Promise**를 반환합니다.
- **Promise**의 **.then()**과 **.catch()**를 사용해 깔끔하게 비동기 코드를 처리할 수 있습니다.

export 와 import 예제

Default export

mortgage.js

```
const calculateAmortization = () => {};
export const calculateMonthlyPayment = () => {};
export default calculateAmortization;
```

main.js

```
import calculateAmortization from "./mortgage";
import calcAmortization from "./mortgage";
import { default as calcAmortization } from "./mortgage";
import { calculateMonthlyPayment } from "./mortgage";
```

Named export

mortgage.js

```
export const calculateAmortization = () => {};
export const calculateMonthlyPayment = () => {};
```

main.js

```
import * as mortgage from "./mortgage";
    mortgage.calculateAmortization();
import { calculateAmortization } from "./mortgage";
    calculateAmortization();
import { calculateAmortization as calcAmortization } from "./mortgage";
calcAmortization();
```

Vite 기반 Javascript 프로젝트 생성

<https://github.com/vitejs/vite/tree/main/packages/create-vite>

Vite 기반 프로젝트 생성

```
npm create vite@latest
```

Need to install the following packages:

create-vite@7.1.1

Ok to proceed? (y) y

```
> npx
```

```
> create-vite
```

```
|
```

```
◇ Project name:
```

```
| employee-es6-vite
```

```
|
```

```
◇ Select a framework:
```

```
| Vanilla
```

```
|
```

```
◇ Select a variant:
```

```
| JavaScript
```

```
|
```

```
◇ Scaffolding project in C:\5.Front_Reactjs\Employee_Gemini\employee-es6-vite...
```

```
|
```

```
└─ Done. Now run:
```

```
cd employee-es6-vite
```

```
npm install
```

```
npm run dev
```

.env.development

VITE_API_URL=<http://localhost:8080/api>

환경변수명은 VITE_ 로 시작해야 합니다.

환경변수 사용

import.meta.env.VITE_API_URL

<https://stackoverflow.com/questions/70522494/multiple-entry-points-in-vite>

Dev Server 실행

개발 dev mode (development) .env.[mode]

.env.development

npm run dev

Production(운영)에 배포를 위한 Build 명령

운영 prod mode (production)

.env.production

npm run build